

Etape 1

Création du topic client_tickets

```
# Méthode recommandée (indépendante du préfixe du container)
docker compose exec redpanda-0 rpk topic create client_tickets --partitions 3 --replicas 3 -X user=superuser -X pass=secretpassword -X
```

- **3 partitions** → tes tickets sont répartis (hash de la clé) → 3 consumers peuvent traiter en parallèle → beaucoup plus rapide.
- **replicas**
 - `--replicas 1` → pas de réplication → chaque partition existe en 1 seul exemplaire → si le broker qui héberge cette partition meurt → perte de données (pas de backup).
 - `--replicas 3` → chaque partition a 3 copies (1 leader + 2 followers) sur 3 brokers différents → tu peux perdre 2 brokers sans perdre de données.

Générer des tickets clients

```
python client_tickets.py
```

Extrait du terminal :

 alt text

Extrait sur Redpanda UI :

 alt text

Etape 2 - Traiter les données avec PySpark

PySpark?

Apache Spark est un moteur open-source ultra-puissant pour traiter de très gros volumes de données (big data) de façon distribuée (sur plusieurs machines/ordinateurs en parallèle). Il est beaucoup plus rapide que les approches classiques (comme Pandas sur une seule machine) quand on a des téraoctets de données.

Ce que fait PySpark en gros :

- Lire des données massives depuis : fichiers (CSV, Parquet, JSON), bases de données, Kafka/Redpanda, S3, etc.
- Transformer / nettoyer / enrichir les données (filtres, jointures, agrégations, UDFs personnalisées...)
- Faire des calculs distribués : groupBy, count, sum, window functions, machine learning (MLlib), graphes, etc.
- Traiter en batch (tout d'un coup) ou en streaming (données qui arrivent en continu, comme chez toi avec les tickets Redpanda)
- Écrire les résultats : fichier, base de données, nouveau topic Kafka, console, etc.

Installation des dépendances

⚠ **Important** : PySpark dépend fortement de Java et Hadoop. Des versions incompatibles peuvent provoquer des erreurs au lancement (SparkContext, Kafka connector, winutils, etc.). Ce guide décrit une configuration **testée et compatible**.

- **winutils + hadoop 3.3.5** : PySpark utilise certaines fonctions Hadoop, même en local. Sous Windows, cela nécessite l'outil **winutils**.
 - ⚠ **Important** : Pour Windows 11, utilisez la version `3.3.6`
 - <https://github.com/cdarlint/winutils>
 - A ajouter dans le PATH (variable d'environnement) le dossier `\bin`
 - Créer une nouvelle variable système
 - nom : `HADOOP_HOME`
 - valeur de la variable : `chemin vers le dossier (C:\...\hadoop-3.3.5)`
- **JAVA 17** : *Une version incompatible peut empêcher Spark de démarrer.*
 - <https://adoptium.net/fr/temurin/releases?version=17>
 - A ajouter dans le PATH (variable d'environnement) le dossier `\bin`
 - Vérifier :

```
java -version
```

- **Installation Python, PySpark, Kafka-Python et FindSpark :**

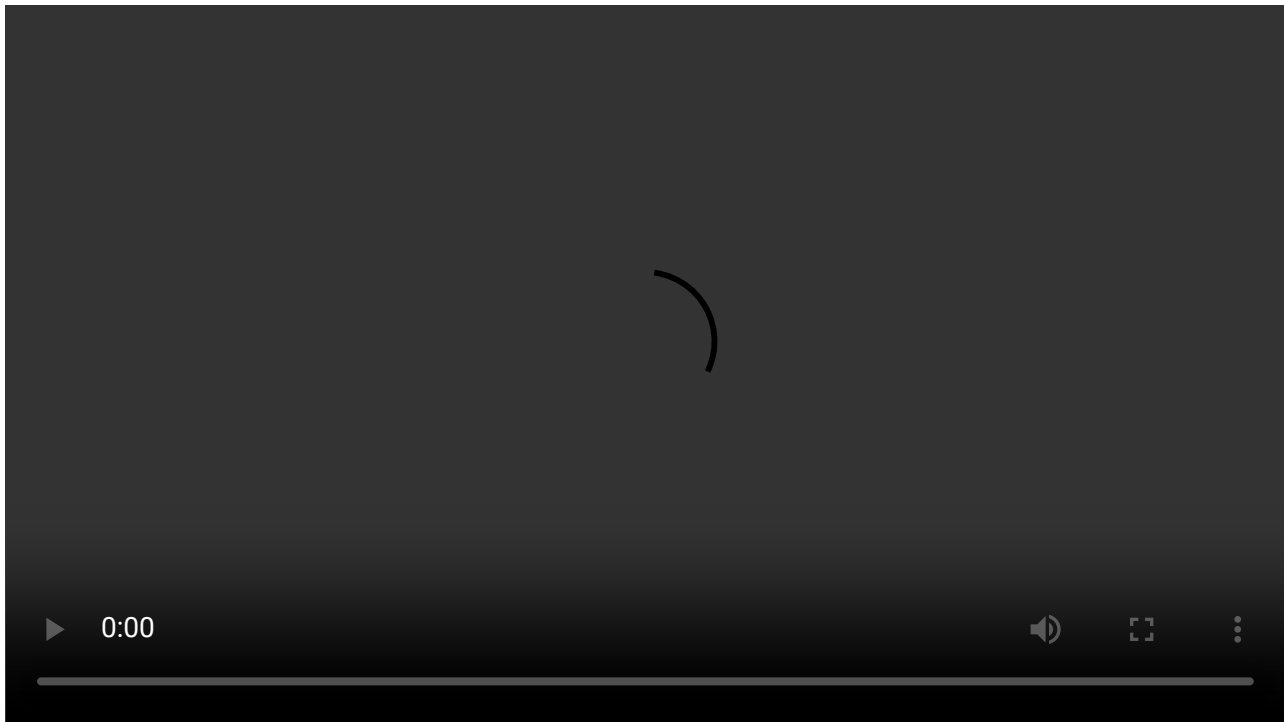
- Python :

```
python v 3.1.3
```

- pip install :

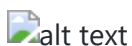
```
kafka-python==2.3.0
pyspark==4.1.1
findspark==2.0.1
```

Etape 3 - Démonstration d'une vidéo présentant la pipeline ETL



Si la vidéo ne se lance pas ou n'a pas de son, vous pouvez la lire avec un autre lecteur. La vidéo se trouve dans le dossier /videos.

Etape 4 - Schéma de l'infrastructure hybride



Etape 5 - Évaluation de compatibilité de l'architecture hybride InduTechData

1. Contexte

InduTechData est une entreprise spécialisée dans l'analyse de données industrielles. Son infrastructure actuelle est composée de :

- Cluster SQL Server de 40 To (ERP / CRM)
- SAN de 10 To pour données non structurées
- Active Directory pour la gestion des identités
- ERP et CRM dédiés

Avec l'arrivée de données IoT (50 Go/mois) et de flux temps réel, l'entreprise souhaite moderniser sa gestion des données via une architecture **hybride cloud/on-premise**, tout en assurant compatibilité, sécurité et maîtrise des coûts.

2. Composants cloud sélectionnés et justification

Composant	Rôle	Justification
Redpanda Cluster (EC2/EKS)	Ingestion et traitement temps réel des flux IoT et logs	Compatible Kafka API, faible latence, scalable horizontalement, simplifie le traitement streaming sans Zookeeper, s'intègre facilement avec SQL Server via connecteurs.
Amazon S3 (Data Lake)	Stockage durable et économique des données IoT et logs	Stockage quasi illimité, coût faible, compatible avec outils BI/analytics. Permet archivage et récupération à la demande.
EC2 / EKS (Analytique)	Traitement et analyse des données	Permet d'héberger ETL, microservices et jobs analytiques, consommation des flux Redpanda et stockage S3.
AWS IAM + AD Connector	Gestion des identités et contrôle d'accès	Fédère Active Directory on-premise avec le cloud, SSO, politique de sécurité homogène.
AWS CloudWatch	Supervision et alerting	Monitorer Redpanda, EC2/EKS et flux de données, alerte en cas d'erreur ou de surcharge.

3. Évaluation de compatibilité

3.1 Sécurité et conformité

- Flux de données sensibles :
 - Redpanda prend en charge **TLS/SSL** pour le transport des messages.
 - Les connexions SQL Server → Cloud peuvent être sécurisées via **VPN ou Direct Connect**.
 - Les données stockées sur S3 peuvent être chiffrées (SSE-S3 ou SSE-KMS).
- Gestion des identités :
 - AD Connector permet une fédération homogène des identités entre Active Directory on-premise et AWS IAM.
 - Les utilisateurs continuent d'utiliser leurs comptes AD, garantissant **cohérence et contrôle centralisé**.

Conclusion sécurité : L'architecture hybride protège correctement les flux sensibles et maintient une gestion unifiée des identités.

3.2 Interopérabilité

- **Redpanda ↔ SQL Server :**
 - Connecteurs Kafka permettent de consommer des tables SQL Server ou d'alimenter SQL Server depuis le cloud.
 - ETL batch ou streaming possible pour synchroniser les données critiques.
- **Automatisation des flux :**
 - Redpanda supporte les pipelines temps réel pour logs et IoT.
 - EC2/EKS peut orchestrer ETL pour SQL Server → S3.
 - Les flux peuvent être planifiés ou déclenchés en fonction d'événements (Kafka Connect + CloudWatch + Lambda).

Conclusion interopérabilité : Redpanda et les services cloud choisis peuvent être intégrés de manière fluide avec le SI existant, avec automatisation possible.

3.3 Scalabilité et gestion des coûts

- **Scalabilité :**
 - Redpanda scale horizontalement pour absorber la croissance des flux IoT.
 - S3 offre un stockage quasi illimité pour logs et données historiques.
 - EC2/EKS permet d'ajuster la puissance de calcul en fonction des besoins analytiques.
- **Surveillance des coûts :**
 - AWS CloudWatch + Budgets AWS permettent de suivre l'utilisation des services et d'alerter en cas de dépassement.
 - Optimisation possible avec EC2 Spot, S3 Intelligent-Tiering et scaling automatique Redpanda.
- **Estimation des coûts initiaux et récurrents (approx.) :**
 - **Redpanda Cluster :** 3–5 nœuds EC2 m5.large → 600–800 € / mois
 - **S3 stockage 50 Go/mois + 10 To historique** → ~20–25 € / mois
 - **EC2/EKS pour analytics :** 2–3 instances → 200–400 € / mois
 - **AWS IAM / AD Connector :** 30–50 € / mois
 - **CloudWatch monitoring :** 50–100 € / mois
 - **Total récurrent approximatif :** 900–1 400 € / mois
 - **Investissement initial :** principalement configuration Redpanda, VPN / Direct Connect (~500–1 000 €)

Ces chiffres sont indicatifs et peuvent être affinés via **AWS Pricing Calculator**.

Etape 6 - Conteneurisation du projet

1. Objectif

Créez un fichier Dockerfile pour chaque élément du projet :

- L'image Docker de Redpanda

- Le script générateur de tickets

- Le script du traitement PySpark

Utilisez Docker Compose pour orchestrer, construire et lancer les conteneurs et volumes de votre projet.

2. Architecture du projet

```
projet/ ├── docker-redpanda/
|   ├── docker-compose-app.yml ← ton fichier existant Redpanda
|   └── project/
|       ├── Dockerfile.generator
|       ├── Dockerfile.stream
|       ├── client_tickets.py
|       ├── clients_tickets_stream.py
|       ├── requirements.txt
|       └── docker-compose.yml
└── exports/ ← dossier pour les JSON (optionnel)
    └── README.md
```

3. Commande Docker

```
docker compose -f docker-redpanda/docker-compose-app.yml -f project/docker-compose.yml up --build
```